

TouchGuide

从 Base Policy 到 Touch Guidance 的训练与推导

–作者: Yang kunpeng

核心主线：先训练视觉生成式策略，再训练 CPM，最后在推理时用
action-gradient steering

前言

这份笔记只沿一条主线展开 TouchGuide: 第一阶段先得到已经训练好的生成式 **base policy**; 第二阶段单独训练 **Contact Physical Model (CPM)**, 并把 CPM 与 **Classifier Guidance** 的数学关系推清楚; 第三阶段冻结所有网络参数, 只对当前生成中的动作求梯度, 在 **inference time** 修改 **Diffusion** 的噪声预测或 **Flow Matching** 的速度场。

写法上尽量把“公式从哪里来”与“公式在算法里做什么”放在一起。每遇到一个容易混淆的位置, 正文会用“问题”“直觉”“易错点”单独提示; 不熟悉的读者可以沿推导逐步阅读, 已经熟悉 **Diffusion / Flow Matching** 的读者也可以直接跳到第二章。

全文主线。

expert demonstrations $\longrightarrow$ base Flow / Diffusion policy
 $\longrightarrow$ CPM: $(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t) \mapsto \mathbf{s}$
 $\longrightarrow \nabla_{\mathbf{A}_t^k} \mathbf{s}$
 $\longrightarrow$ inference-time steering of u_θ or ϵ_θ .

符号约定

符号	含义
$\mathbf{V}_t$	机器人在真实控制时刻 t 的视觉观测
$\mathbf{T}_t$	机器人在真实控制时刻 t 的触觉观测
$\mathbf{A}_t^k$	控制时刻 t 内部，第 k 个 Diffusion / Flow Matching 采样步的中间动作
$\mathbf{A}_t^0$	当前控制时刻最终生成并执行的动作 / action chunk
k	内层生成步骤；在 flow-matching 公式中还会被 scale 到 $[1, 0]$ 的连续生成时间
u_θ	Flow Matching base policy 预测的 velocity field
ϵ_θ	Diffusion Policy 预测的 noise
$\mathbf{O}_t$	视觉-触觉 observation latent
$\mathbf{a}_t$	action latent
$\mathbf{s} = s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t)$	CPM 输出的标量 feasibility / compatibility score
η	guidance scale，控制 CPM gradient 对 base policy 的影响强度
$K_{\text{TouchGuide}}$	采样后期启用 TouchGuide 的 guidance steps 数量

t 与 k 的区别。 t 是机器人与真实环境交互的外层控制时间； k 是在固定 t 下生成 $\mathbf{A}_t^0$ 时的内层采样时间。因此一个控制时刻内部发生的是

$$\mathbf{A}_t^K \rightarrow \mathbf{A}_t^{K-1} \rightarrow \dots \rightarrow \mathbf{A}_t^0,$$

执行 $\mathbf{A}_t^0$ 后，环境才进入 $t + 1$ 。

目录

前言	ii
符号约定	iii
第一章 第一阶段：训练 Base Policy	1
1.1 $\pi_{0.5}$ 的 Flow Matching: 训练一个 velocity field	1
1.2 Diffusion Policy: 训练一个 noise predictor	2
1.3 Flow Matching 与 Diffusion Policy 对比	3
1.4 阶段切换: 哪些参数冻结?	3
第二章 第二阶段：训练 CPM 与推导 Touch Guidance	4
2.1 CPM 网络结构	4
2.2 CPM 的对比学习 loss	5
2.3 为什么 CPM 训练时也要给 action 加噪声?	6
2.4 Flow Matching 的 classifier guidance 推导	6
2.5 Diffusion Policy 的 classifier guidance 推导	9
2.6 为什么 CPM 可以替代 classifier guidance 中的 classifier gradient? . .	11
2.7 把 CPM gradient 代入两类 base policy	13
第三章 第三阶段：推理时的 TouchGuide	14
3.1 TouchGuide 推理算法	14
3.2 训练与推理的参数状态总结	16
整条主线总结	17

第一阶段：训练 Base Policy

问题：第一阶段到底训练什么？

TouchGuide 的核心前提是 base policy 已经训练完成。它对 pre-trained diffusion-based 或 flow-matching-based visuomotor policy 做 inference-time steering，而不是把触觉重新并入大策略联合训练 [1]。

1.1 $\pi_{0.5}$ 的 Flow Matching：训练一个 velocity field

在 TouchGuide 中， $\pi_{0.5}$ 的 action expert 是 flow-matching based。为了理解训练目标，先看最常用的线性 probability path：

$$\mathbf{A}^k = (1 - k)\mathbf{A}^0 + k\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad k \in [0, 1]. \quad (1.1)$$

这里 $\mathbf{A}^0$ 是专家数据中的 clean action， ϵ 是高斯噪声。 $k = 0$ 对应 clean action， $k = 1$ 对应 noise。对 k 求导：

$$\frac{d\mathbf{A}^k}{dk} = \epsilon - \mathbf{A}^0. \quad (1.2)$$

因此，一个标准 conditional flow-matching action expert 的训练可以写成 velocity regression：

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\mathbf{A}^0, \epsilon, k} \left[\left\| u_{\theta}(\mathbf{A}^k, \mathbf{V}, k) - (\epsilon - \mathbf{A}^0) \right\|_2^2 \right]. \quad (1.3)$$

网络 u_{θ} 的任务不是预测最终动作，而是预测当前中间点 $\mathbf{A}^k$ 应沿哪个 velocity 移动。

直觉：Flow Matching 学的是“当前 noisy action 下一步往哪里走”，因此监督对象是 velocity，而不是最终动作本身。

实际 $\pi_{0.5}$ 是 VLA + action expert 架构，TouchGuide 论文在实验中采用官方 $\pi_{0.5}$ 实现和 pi05_base 权重进行任务微调；action expert 仍然按 flow matching 生成连续动作 chunk[7, 1]。

对于本文关心的 TouchGuide，最重要的不是 $\pi_{0.5}$ 的 VLM 细节，而是：训练完成后我们拥有一个 velocity predictor

$$u_{\theta}(\mathbf{A}_t^k, \mathbf{V}_t),$$

在 TouchGuide 第二阶段和推理阶段都不需要重新定义它的训练 loss。

1.2 Diffusion Policy：训练一个 noise predictor

Diffusion Policy 使用 DDPM 风格的前向加噪。对于专家动作 $\mathbf{A}^0$ ，第 k 个噪声层级的中间动作满足

$$\mathbf{A}^k = \sqrt{\bar{\alpha}_k} \mathbf{A}^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (1.4)$$

其中通常定义 $\alpha_k = 1 - \beta_k$ ，以及

$$\bar{\alpha}_k = \prod_{i=1}^k \alpha_i.$$

$\bar{\alpha}_k$ 越小，说明当前 $\mathbf{A}^k$ 中保留的 clean-action 成分越少、噪声越多。

TouchGuide 使用的是 epsilon-prediction Diffusion Policy。训练目标是让网络根据 noisy action、视觉条件和 diffusion step 预测加进去的噪声：

$$\mathcal{L}_{\text{DP}}(\theta) = \mathbb{E}_{\mathbf{A}^0, \epsilon, k} \left[\left\| \epsilon - \epsilon_{\theta}(\mathbf{A}^k, \mathbf{V}, k) \right\|_2^2 \right]. \quad (1.5)$$

所以 Flow Matching 与 Diffusion 的根本差别，是 base network 学的目标不同：前者预测 velocity，后者预测 noise。

直觉：两者都从噪声逐步得到动作，但 Flow Matching 直接学习速度场；Diffusion Policy 则学习当前样本里包含的噪声。

1.3 Flow Matching 与 Diffusion Policy 对比

表 1.1: 第一阶段 base policy 的核心对比。TouchGuide 不修改这些 base-policy loss。

对比项	Flow Matching ($\pi_{0.5}$ action expert)	Diffusion Policy
起点/终点	通过 probability path 连接 noise 与 expert action	通过 DDPM forward process 将 expert action 逐步加噪
中间状态	$\mathbf{A}^k = (1-k)\mathbf{A}^0 + k\epsilon$ (线性 path 的代表形式)	$\mathbf{A}^k = \sqrt{\bar{\alpha}_k}\mathbf{A}^0 + \sqrt{1 - \bar{\alpha}_k}\epsilon$
网络预测量	velocity u_θ	noise ϵ_θ
监督 target	$\epsilon - \mathbf{A}^0$ (对线性 path 求导)	加入的真实噪声 ϵ
典型 loss	velocity MSE	noise MSE
推理更新	沿 ODE velocity field 积分	使用 scheduler 做 reverse denoising
TouchGuide 改什么	推理时修改 u_θ	推理时修改 ϵ_θ
TouchGuide 是否重训 base policy	否	否

1.4 阶段切换：哪些参数冻结？

第一阶段完成后，base-policy 参数 θ 被视为固定。实验中，DP 训练 300 epochs； $\pi_{0.5}$ 使用 pi05_base，不同任务做 20k-40k steps 的任务训练/微调。进入 CPM 训练后，不把 $\mathcal{L}_{\text{FM}}$ 或 $\mathcal{L}_{\text{DP}}$ 与 CPM loss 联合优化 [1]。

第二阶段：训练 CPM 与推导 Touch Guidance

2.1 CPM 网络结构

问题：CPM 网络到底输出什么？

CPM 的输入是当前视觉 $\mathbf{V}_t$ 、触觉 $\mathbf{T}_t$ 与候选动作 $\mathbf{A}_t$ 。它不是直接输出动作，而是输出一个标量 feasibility score。

CPM 使用 pretrained DINOv2 同时编码视觉和光学触觉图像，得到 $\mathbf{T}_t^{\text{emb}}$ 和 $\mathbf{V}_t^{\text{emb}}$ 。随后用 N 层 Transformer Encoder 融合两种 observation embedding，并做 L2 normalization：

$$\mathbf{O}_t^{\text{emb}} = \text{TranEnc} \left(\text{concat} [\mathbf{T}_t^{\text{emb}}, \mathbf{V}_t^{\text{emb}}] \right)_{\times N} \xrightarrow{\text{L2 Norm}} \mathbf{O}_t. \quad (2.1)$$

动作分支用 action encoder $\mathcal{E}_A$ (1D CNN + MLP)：

$$\mathbf{A}_t^{\text{emb}} = \mathcal{E}_A(\mathbf{A}_t) \xrightarrow{\text{L2 Norm}} \mathbf{a}_t. \quad (2.2)$$

最后 feasibility score 是 observation latent 与 action latent 的余弦相似度：

$$\mathbf{s} = s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t) = \mathbf{O}_t^\top \mathbf{a}_t. \quad (2.3)$$

因为两边都做 L2 normalization, $\|\mathbf{O}_t\|_2 = \|\mathbf{a}_t\|_2 = 1$ ，所以点积就是 cosine similarity。

直觉：CPM 不是在回归力、摩擦系数或动力学参数；它学习的是“当前视觉-触觉状态与候选动作是否匹配”的兼容性分数。

消融中，视觉和触觉两侧均采用 pretrained DINOv2 并冻结时效果最好；因此 CPM 训练时，DINOv2 冻结，而后续 Transformer fusion、action encoder 以及 learnable temperature τ 参与训练 [1]。

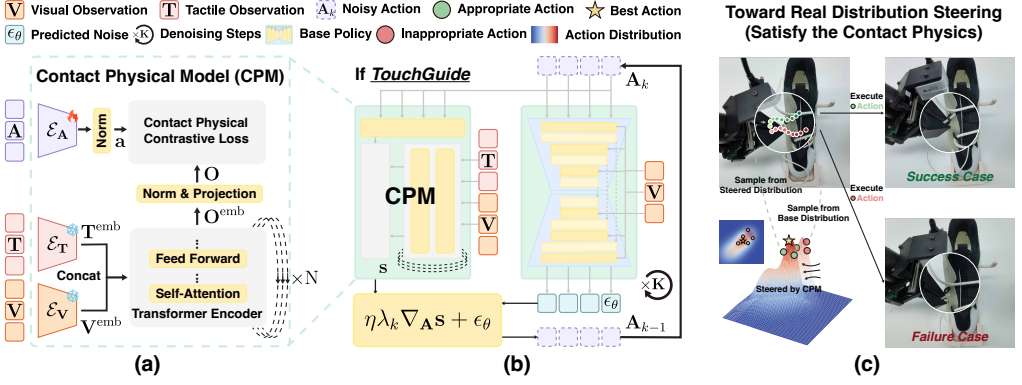


图 2.1: CPM 结构与 inference-time steering。左侧为视觉-触觉 observation 与 action 的编码及匹配，中间为推理时的 gradient steering，右侧展示 action distribution 向 real distribution 靠近的直觉。

2.2 CPM 的对比学习 loss

对于一个 batch 中的 M 个 expert samples，记其 latent observation 和 latent action 为

$$\{(\mathbf{O}_i, \mathbf{a}_i)\}_{i=1}^M.$$

同一时间步的 $(\mathbf{O}_i, \mathbf{a}_i)$ 是正样本；来自不同时间步的错配，例如 $(\mathbf{O}_{t+\Delta t}, \mathbf{a}_t)$ 或 $(\mathbf{O}_t, \mathbf{a}_{t+\Delta t})$ ，构成负样本。训练采用双向 contrastive loss：

$$\mathcal{L}_{\text{CPM}}(\mathbf{O}_t, \mathbf{a}_t) = \frac{1}{2} (\mathcal{L}_{\mathbf{O} \rightarrow \mathbf{a}} + \mathcal{L}_{\mathbf{a} \rightarrow \mathbf{O}}), \quad (2.4)$$

其中

$$\mathcal{L}_{\mathbf{O} \rightarrow \mathbf{a}} = -\frac{1}{M} \sum_{i=1}^M \log \frac{\exp(\mathbf{O}_i^\top \mathbf{a}_i / \tau)}{\sum_{j=1}^M \exp(\mathbf{O}_i^\top \mathbf{a}_j / \tau)}, \quad (2.5)$$

$$\mathcal{L}_{\mathbf{a} \rightarrow \mathbf{O}} = -\frac{1}{M} \sum_{i=1}^M \log \frac{\exp(\mathbf{a}_i^\top \mathbf{O}_i / \tau)}{\sum_{j=1}^M \exp(\mathbf{a}_i^\top \mathbf{O}_j / \tau)}. \quad (2.6)$$

这里 τ 是 learnable temperature， M 是 batch 中样本数。第一项可以读作“给定 observation，找出正确 action”；第二项则反过来“给定 action，找出正确 observation”。

直觉： 把一个 batch 的相似度写成矩阵后，训练目标就是让正确配对落在对角线上并取得更高相似度。

2.3 为什么 CPM 训练时也要给 action 加噪声？

推理时 CPM 看到的不是 clean expert action，而是 base policy 采样中的中间动作 $\mathbf{A}_t^k$ 。所以 CPM 训练时也主动把 ground-truth action 扰动到 noisy action space：对于 DP，使用与 DP 相同的 noise scheduler，并按几何分布采样 noising level；对于 flow-matching policy，则使用线性 interpolation。这样 CPM 训练时看到的 action 分布与 inference-time guidance 时更一致 [1]。

2.4 Flow Matching 的 classifier guidance 推导

问题：Flow Matching 中的 $k/(1-k)$ 从哪里来？

先从 Gaussian probability path 开始：

$$p_t(x | x_0) = \mathcal{N}(x | \alpha_t x_0, \sigma_t^2 I). \quad (2.7)$$

重参数化为

$$x = \alpha_t x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2.8)$$

对时间 t 求导，并把 $\epsilon = (x - \alpha_t x_0)/\sigma_t$ 代回，可得到 conditional vector field：

$$u_t(x | x_0) = \frac{\sigma'_t}{\sigma_t} x + \left(\alpha'_t - \frac{\sigma'_t \alpha_t}{\sigma_t} \right) x_0. \quad (2.9)$$

Gaussian path 的 score 为

$$\nabla_x \log p_t(x | x_0) = -\frac{x - \alpha_t x_0}{\sigma_t^2}. \quad (2.10)$$

把 x_0 用上式消掉，得到

$$u_t(x | x_0) = a_t x + b_t \nabla_x \log p_t(x | x_0), \quad (2.11)$$

其中

$$a_t = \frac{\alpha'_t}{\alpha_t}, \quad b_t = \left(\frac{\alpha'_t}{\alpha_t} - \frac{\sigma'_t}{\sigma_t} \right) \sigma_t^2. \quad (2.12)$$

对 x_0 边缘化后，得到

$$u_t(x) = a_t x + b_t \nabla_x \log p_t(x). \quad (2.13)$$

因此如果网络已经学到 $u_\theta(x_t) \approx u_t(x_t)$ ，那么它等价地隐含了当前 probability path 的 score。

从 conditional score 到 guided velocity

现在希望生成结果满足条件 y 。原来无条件的 probability path 使用

$$p_t(x_t),$$

而加入条件以后，目标分布变成

$$p_t(x_t | y).$$

因此，原来的 score

$$\nabla_{x_t} \log p_t(x_t)$$

也要换成 conditional score

$$\nabla_{x_t} \log p_t(x_t | y).$$

先从 Bayes 公式开始：

$$p_t(x_t | y) = \frac{p_\phi(y | x_t) p_t(x_t)}{p_t(y)}. \quad (2.14)$$

两边取对数：

$$\log p_t(x_t | y) = \log p_t(x_t) + \log p_\phi(y | x_t) - \log p_t(y). \quad (2.15)$$

再对当前状态 x_t 求梯度。由于 $p_t(y)$ 与 x_t 无关，

$$\nabla_{x_t} \log p_t(y) = 0,$$

所以

$$\nabla_{x_t} \log p_t(x_t | y) = \nabla_{x_t} \log p_t(x_t) + \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.16)$$

这一步可以直接读成

$$\text{conditional score} = \text{unconditional score} + \text{classifier guidance score}.$$

其中

$$\nabla_{x_t} \log p_\phi(y | x_t)$$

表示：如果稍微改变当前 x_t ，往哪个方向移动会让条件 y 的 log-probability 增长最快。

接下来把 conditional score 放回 velocity-score 关系。无条件 velocity 为

$$u_t(x_t) = a_t x_t + b_t \nabla_{x_t} \log p_t(x_t). \quad (2.17)$$

满足条件 y 时，对应的 velocity 写成

$$\hat{u}_t(x_t | y) = a_t x_t + b_t \nabla_{x_t} \log p_t(x_t | y). \quad (2.18)$$

将式 (2.16) 代入：

$$\hat{u}_t(x_t | y) = a_t x_t + b_t [\nabla_{x_t} \log p_t(x_t) + \nabla_{x_t} \log p_\phi(y | x_t)] \quad (2.19)$$

$$= \underbrace{[a_t x_t + b_t \nabla_{x_t} \log p_t(x_t)]}_{u_t(x_t)} + b_t \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.20)$$

括号中的前两项正好就是原来的无条件 velocity，因此

$$\hat{u}_t(x_t | y) = u_t(x_t) + b_t \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.21)$$

实际网络用 u_θ 近似 u_t ，于是写成

$$\hat{u}_\theta(x_t) := u_\theta(x_t) + b_t \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.22)$$

直觉：这里不是“凭空给 velocity 加一个梯度”。先用 Bayes 得到 conditional score 比 unconditional score 多出的那一项，再利用 velocity 与 score 的线性关系，把这一个 score 修正乘上 b_t 转成 velocity 修正。也就是说， b_t 的作用就是把 score-space 中的 guidance 换算到 velocity-space。

TouchGuide 使用 optimal-transport linear path

$$\alpha_t = 1 - t, \quad \sigma_t = t.$$

于是

$$\frac{\alpha'_t}{\alpha_t} = -\frac{1}{1-t}, \quad \frac{\sigma'_t}{\sigma_t} = \frac{1}{t}.$$

直接代入 b_t :

$$b_t = \left(-\frac{1}{1-t} - \frac{1}{t} \right) t^2 \quad (2.23)$$

$$= -\frac{1}{t(1-t)} t^2 \quad (2.24)$$

$$= -\frac{t}{1-t}. \quad (2.25)$$

再加入 guidance scale η :

$$\hat{u}_\theta(x_t) = u_\theta(x_t) - \eta \frac{t}{1-t} \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.26)$$

因此 $t/(1-t)$ 不是额外学习的参数，而是由所选 flow path 的 α_t, σ_t 推出的时间系数。

易错点：这里的 t 是 Flow Matching 的连续生成时间；在 TouchGuide Algorithm 1 中离散采样索引 k 会先被 scale 到 $[1, 0]$ ，随后论文继续用 k 表示这个归一化时间。

2.5 Diffusion Policy 的 classifier guidance 推导

问题：Diffusion 中的 $\sqrt{1-\bar{\alpha}_t}$ 为什么会出现在 guidance 前面？

先看 classifier 如何改变 reverse diffusion 的 Gaussian 均值，再把结果改写成 epsilon-prediction 形式。

Dhariwal–Nichol：条件 reverse transition 的均值为什么会被 classifier gradient 平移？

Dhariwal–Nichol 从一个已经训练好的 unconditional reverse transition 出发：

$$p_\theta(x_t | x_{t+1}) = \mathcal{N}(x_t; \mu, \Sigma). \quad (2.27)$$

若希望采样结果满足条件 y ，其论文写出条件 reverse transition（差一个归一化常数 Z ）：

$$p_{\theta, \phi}(x_t | x_{t+1}, y) = Z p_{\theta}(x_t | x_{t+1}) p_{\phi}(y | x_t). \quad (2.28)$$

这里 $p_{\phi}(y | x_t)$ 就是额外训练在 noisy sample 上的 classifier。

对式 (2.27) 取 log：

$$\log p_{\theta}(x_t | x_{t+1}) = -\frac{1}{2}(x_t - \mu)^{\top} \Sigma^{-1}(x_t - \mu) + C. \quad (2.29)$$

Dhariwal–Nichol 在 $x_t = \mu$ 附近对 classifier 的 log probability 做一阶 Taylor 展开：

$$\log p_{\phi}(y | x_t) \approx (x_t - \mu)^{\top} g + C_1, \quad g = \nabla_{x_t} \log p_{\phi}(y | x_t)|_{x_t=\mu}. \quad (2.30)$$

将两项相加并配平方：

$$\log[p_{\theta}(x_t | x_{t+1}) p_{\phi}(y | x_t)] \quad (2.31)$$

$$\approx -\frac{1}{2}(x_t - \mu)^{\top} \Sigma^{-1}(x_t - \mu) + (x_t - \mu)^{\top} g + C_2 \quad (2.32)$$

$$= -\frac{1}{2}(x_t - \mu - \Sigma g)^{\top} \Sigma^{-1}(x_t - \mu - \Sigma g) + C_3. \quad (2.33)$$

因此，条件 reverse transition 仍近似为同协方差的 Gaussian，只是均值由

$$\mu \longrightarrow \mu + \Sigma g$$

平移。加入 gradient scale η 后，就是

$$x_t \sim \mathcal{N}(\mu + \eta \Sigma \nabla_{x_t} \log p_{\phi}(y | x_t), \Sigma). \quad (2.34)$$

这就是“classifier guidance”最直接的含义：**classifier 对当前 noisy sample 的输入梯度，平移 reverse diffusion 的采样均值。**[2]

为什么 TouchGuide 写成修改 ϵ_{θ} ？

TouchGuide 沿用 epsilon-prediction 的记号。DDPM 前向扰动写成

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2.35)$$

给定 x_0 , Gaussian conditional score 为

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{1 - \bar{\alpha}_t} \quad (2.36)$$

$$= -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}}. \quad (2.37)$$

因此 epsilon-prediction 与 score 的换算关系为

$$\epsilon = -\sqrt{1 - \bar{\alpha}_t} \text{ score}. \quad (2.38)$$

另一方面, 由 Bayes, conditional score 可以拆成

$$\nabla_{x_t} \log p_t(x_t | y) = \nabla_{x_t} \log p_t(x_t) + \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.39)$$

将第二项乘上 guidance scale η , 再使用式 (2.38) 从 score 换回 epsilon prediction, 可得

$$\hat{\epsilon}_\theta(x_t) = \epsilon_\theta(x_t) - \eta \sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p_\phi(y | x_t). \quad (2.40)$$

于是得到 diffusion classifier-guidance 的 epsilon-prediction 形式。

直觉: Classifier Guidance 首先修改的是条件 score; 由于 epsilon-prediction 与 score 相差一个 $-\sqrt{1 - \bar{\alpha}_t}$ 的换算关系, guidance 写回噪声预测时自然带出这个时间系数。

这里的 $\sqrt{1 - \bar{\alpha}_t}$ 不是额外经验参数, 而是把 classifier 提供的 *score-space gradient* 换算回 epsilon-prediction 时自然出现的噪声尺度。

2.6 为什么 CPM 可以替代 classifier guidance 中的 classifier gradient ?

问题: CPM 明明只是对比学习得到的 cosine score, 为什么能放到 Classifier Guidance 的位置?

这里要区分两件事: **CPM 不是真正的二分类器, 也不严格输出概率。**它只是通过一个近似的概率解释, 把 CPM score 的梯度和 classifier-guidance gradient 联系起来。

设 $\mathbf{a}_{\text{real}}$ 是来自 real distribution $\mathcal{Q}_{\text{real}}$ 的 optimal-action latent, $\mathbf{a}$ 是当前 policy action $\mathbf{A}$ 的 latent。论文假设

$$p(\mathbf{a}_{\text{real}} | \mathbf{a}) = \mathcal{N}(\mathbf{a}_{\text{real}} | \mu = \mathbf{a}, \Sigma = \sigma I). \quad (2.41)$$

因此

$$\log p(\mathbf{a}_{\text{real}} | \mathbf{a}) = -\frac{1}{2\sigma} \|\mathbf{a}_{\text{real}} - \mathbf{a}\|_2^2 + C. \quad (2.42)$$

CPM 对比学习的目标, 是把当前 observation latent $\mathbf{O}$ 与对应的 real/expert action latent 对齐, 因此近似有

$$\mathbf{O} \approx \mathbf{a}_{\text{real}}.$$

于是

$$\log p(\mathbf{a}_{\text{real}} | \mathbf{a}) \approx -\frac{1}{2\sigma} \|\mathbf{O} - \mathbf{a}\|_2^2 + C \quad (2.43)$$

$$= -\frac{1}{2\sigma} \left(\|\mathbf{O}\|_2^2 + \|\mathbf{a}\|_2^2 - 2\mathbf{O}^\top \mathbf{a} \right) + C. \quad (2.44)$$

因为 L2 normalization 给出 $\|\mathbf{O}\|_2 = \|\mathbf{a}\|_2 = 1$, 对真正动作输入 $\mathbf{A}$ 求梯度时, 常数项消失:

$$\nabla_{\mathbf{A}} \log p(\mathbf{a}_{\text{real}} | \mathbf{a}) = \frac{1}{\sigma} \nabla_{\mathbf{A}} \mathbf{O}^\top \mathbf{a} = \frac{1}{\sigma} \nabla_{\mathbf{A}} \mathbf{s}. \quad (2.45)$$

$1/\sigma$ 可以吸收到 guidance scale η 中。因此, 标准 classifier guidance 需要的

$$\nabla_{\mathbf{A}} \log p_{\phi}(y | \mathbf{A})$$

在 TouchGuide 中由

$$\nabla_{\mathbf{A}} \mathbf{s} = \nabla_{\mathbf{A}} s_{\phi}(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A})$$

近似替代。

直觉: 这一步不是在声称 CPM 等于一个概率分类器, 而是在说明: 在高斯 latent 假设和 L2 normalization 下, CPM 标量分数的 action-gradient 与 “靠近真实动作” 的 log-likelihood gradient 只差一个常数比例。

这里的梯度是对 **action 输入求梯度**, 不是对 CPM 参数求梯度。推理阶段 ϕ 冻结, $\mathbf{A}_t^k$ 设为可求导变量, autograd 沿

$$\mathbf{A}_t^k \rightarrow \mathcal{E}_{\mathbf{A}} \rightarrow \mathbf{a}_t \rightarrow \mathbf{O}_t^\top \mathbf{a}_t$$

反向传播，得到与 $\mathbf{A}_t^k$ 同 shape 的 $\nabla_{\mathbf{A}_t^k} \mathbf{s}$ 。

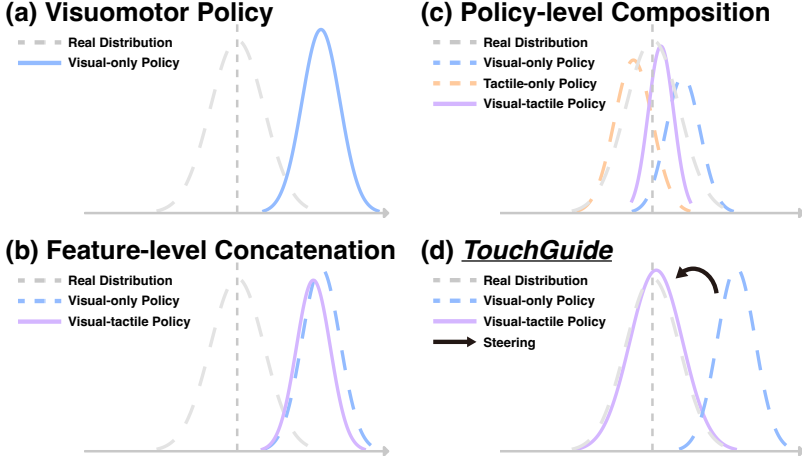


图 2.2: CPM steering 的分布直觉：将 $\mathcal{Q}_{\text{policy}}$ 向 $\mathcal{Q}_{\text{real}}$ 推动。

2.7 把 CPM gradient 代入两类 base policy

对 epsilon-prediction Diffusion Policy, TouchGuide 使用

$$\hat{\epsilon}_\theta = \epsilon_\theta(\mathbf{A}_t^k, \mathbf{V}_t) - \eta \sqrt{1 - \bar{\alpha}_k} \nabla_{\mathbf{A}_t^k} s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t^k). \quad (2.46)$$

对 flow-matching-based $\pi_{0.5}$ action expert, TouchGuide 使用

$$\hat{u}_\theta = u_\theta(\mathbf{A}_t^k, \mathbf{V}_t) - \eta \frac{k}{1-k} \nabla_{\mathbf{A}_t^k} s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t^k). \quad (2.47)$$

这里 Flow Matching 的 k 在 Algorithm 1 中先从离散采样索引 $[K, 0]$ scale 到连续区间 $[1, 0]$ 。论文实现 $\pi_{0.5}$ 时采用 $1 \rightarrow 0$ 的积分方向，即 $dt < 0$ ，因此 velocity 公式中的负号与反向积分结合后，实际 action update 仍然朝 feasibility score 增大的方向移动 [1]。

第三阶段：推理时的 TouchGuide

问题：推理时到底谁在更新？

推理时没有 optimizer step：base policy θ 冻结，CPM ϕ 冻结，DINOv2 也冻结；唯一需要梯度的是当前中间动作 $\mathbf{A}_t^k$ 。采样分成两个阶段：早期只让视觉 base policy 形成 coarse visually plausible action，后期才启动 CPM touch guidance。

3.1 TouchGuide 推理算法

在每个控制时刻 t ，从高斯噪声 $\mathbf{A}_t^K$ 开始逐步生成动作；仅在后 $K_{\text{TouchGuide}}$ 个采样步骤启用 CPM guidance。

Algorithm 1 TouchGuide (Inference Time)

Require: Base policy π_θ , Contact Physical Model s_ϕ , total steps K , guidance steps $K_{\text{TouchGuide}}$, guidance scale η , visual observation $\mathbf{V}_t$, tactile observation $\mathbf{T}_t$

```

1:  $\mathbf{A}_t^K \leftarrow \text{Sample from } \mathcal{N}(0, I)$ 
2: for  $k = K, \dots, 1$  do
3:   if  $\pi_\theta \in \text{Diffusion-based}$  then ▷ e.g., DP, DP3
4:      $\epsilon \leftarrow \pi_\theta(\mathbf{V}_t)$ 
5:     if  $k \leq K_{\text{TouchGuide}}$  then
6:        $\mathbf{s} \leftarrow s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t^k)$ 
7:        $\hat{\epsilon} \leftarrow \epsilon - \eta \sqrt{1 - \bar{\alpha}_k} \nabla_{\mathbf{A}_t^k} \mathbf{s}$ 
8:        $\mathbf{A}_t^{k-1} \leftarrow \text{Update } \mathbf{A}_t^k \text{ using } \hat{\epsilon}$ 
9:     else
10:       $\mathbf{A}_t^{k-1} \leftarrow \text{Update } \mathbf{A}_t^k \text{ using } \epsilon$ 
11:    end if
12:  else if  $\pi_\theta \in \text{Flow-matching}$  then ▷ e.g.,  $\pi_{0.5}$ 
13:     $k \leftarrow \text{Scale } [K, 0] \text{ to } [1, 0]$ 
14:     $u \leftarrow \pi_\theta(\mathbf{V}_t)$ 
15:    if  $k \leq K_{\text{TouchGuide}}$  then
16:       $\mathbf{s} \leftarrow s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t^k)$ 
17:       $\hat{u} \leftarrow u - \eta \frac{k}{1-k} \nabla_{\mathbf{A}_t^k} \mathbf{s}$ 
18:       $\mathbf{A}_t^{k-1} \leftarrow \text{Update } \mathbf{A}_t^k \text{ using } \hat{u}$ 
19:    else
20:       $\mathbf{A}_t^{k-1} \leftarrow \text{Update } \mathbf{A}_t^k \text{ using } u$ 
21:    end if
22:  end if
23: end for
24: return  $\mathbf{A}_t^0$ 

```

3.2 训练与推理的参数状态总结

表 3.1: 三个阶段的参数更新关系。

模块	第一阶段: base policy	第二阶段: CPM	第三阶段: inference
Base policy θ	训练 / 任务微调	冻结	冻结
DINOv2 encoders	–	冻结	冻结
CPM Transformer	–	训练	冻结
CPM action encoder	–	训练	冻结
Temperature τ	–	训练	冻结
当前 action $\mathbf{A}_t^k$	作为训练样本/中间变量	作为 CPM 输入	设为可求导, 计算 $\nabla_{\mathbf{A}_t^k} \mathbf{s}$

整条主线总结

第一阶段学的是 base generation direction: Flow Matching 学 u_θ , Diffusion 学 ϵ_θ 。第二阶段 CPM 用对比学习学一个标量 compatibility / feasibility potential:

$$\mathbf{s} = s_\phi(\mathbf{V}_t, \mathbf{T}_t, \mathbf{A}_t) = \mathbf{O}_t^\top \mathbf{a}_t.$$

上面的近似推导说明

$$\nabla_{\mathbf{A}} \mathbf{s} \propto \nabla_{\mathbf{A}} \log p(\mathbf{a}_{\text{real}} | \mathbf{a}),$$

所以它可以占据 classifier guidance 中条件梯度的位置。第三阶段不再训练任何网络，只在采样后期把这个 gradient 注入 base policy:

$$\text{Diffusion: } \hat{\epsilon}_\theta = \epsilon_\theta - \eta \sqrt{1 - \bar{\alpha}_k} \nabla_{\mathbf{A}_t^k} \mathbf{s},$$

$$\text{Flow Matching: } \hat{u}_\theta = u_\theta - \eta \frac{k}{1-k} \nabla_{\mathbf{A}_t^k} \mathbf{s}.$$

因此 TouchGuide 的算法本质可以概括为: **pre-trained generative visuomotor policy + separately trained differentiable contact-compatibility potential + inference-time gradient steering**。

自检: 若把 CPM 去掉，系统退化为原来的 visual base policy; 若保留 CPM 但不 $\mathbf{A}_t^k$ 求梯度，CPM 只会“打分”而不能真正 steering 动作。

参考文献

- [1] Zhemeng Zhang et al. *TouchGuide: Inference-Time Steering of Visuomotor Policies via Touch Guidance*. arXiv:2601.20239v6, 2026.
- [2] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. NeurIPS, 2021.
- [3] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. *Flow Matching for Generative Modeling*. ICLR, 2023.
- [4] Qinqing Zheng, Matt Le, Neta Shaul, Yaron Lipman, Aditya Grover, and Ricky T. Q. Chen. *Guided Flows for Generative Modeling and Decision Making*. arXiv:2311.13443, 2023.
- [5] Ruiqi Feng et al. *On the Guidance of Flow Matching*. arXiv:2502.02150, 2025.
- [6] Cheng Chi et al. *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*. 2023.
- [7] Kevin Black et al. $\pi_{0.5}$: *A Vision-Language-Action Model with Open-World Generalization*. arXiv:2504.16054, 2025.